

ENSF 380

TOPIC 08: ARRAYS

Arrays

Arrays are not objects

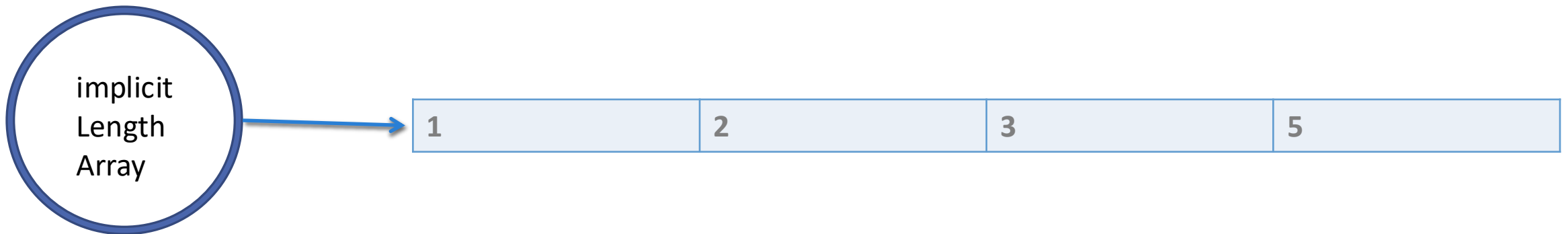
Arrays can contain primitive data types, or references to objects

Default values

- int: 0
- boolean: false
- double/float: 0.0
- char: \u0000 (null character)
- objects: null

One Dimensional Arrays

```
...  
// Create an array of integers containing the values shown  
int[] implicitLengthArray = {1, 2, 3, 5};  
  
// Iterate through the array and output the value.  
for (int tmpValue : implicitLengthArray) {  
    System.out.println(tmpValue);  
}  
...
```



One Dimensional Arrays

```
... // Create an array of integers with a length of three
int[] explicitLengthArray = new int[3];

// Iterate through the array: show original values; set new ones
for (int i=0; i < explicitLengthArray.length; i++) {
    System.out.println(explicitLengthArray[i]);
    explicitLengthArray[i] = 10*i;
    System.out.println(explicitLengthArray[i]);
}
...
```

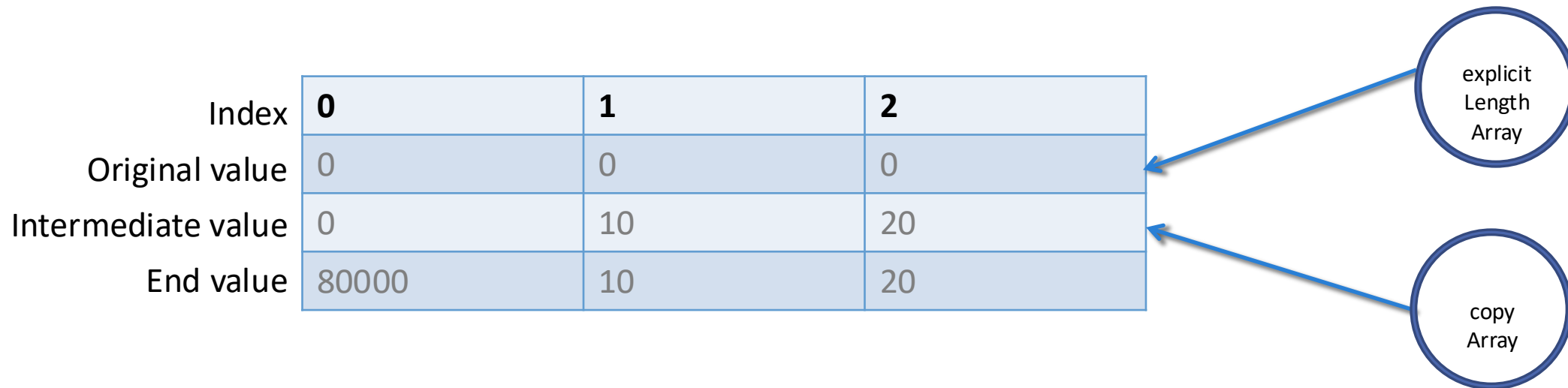
Index	0	1	2
Original value	0	0	0
End value	0	10	20

One Dimensional Arrays

```
...  
// Create an array of integers with a length of three  
int[] explicitLengthArray = new int[3];  
  
// original reference now points to a new array  
explicitLengthArray = new int[2];  
...
```

Array Copies

```
... int[] explicitLengthArray = new int[3];  
  
// Copy an array (by reference)  
int[] copyArray = explicitLengthArray;  
copyArray[0] = 80000;  
... System.out.println(explicitLengthArray[0]);
```



Array Copies

```
import java.util.Arrays;

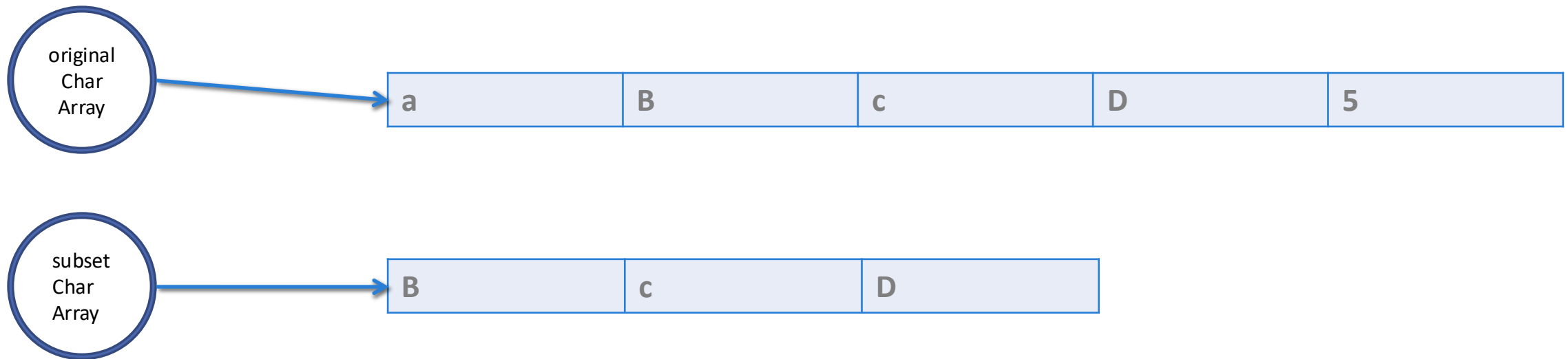
int[] valueCopyArray = Arrays.copyOf(explicitLengthArray, 2);
valueCopyArray[1] = 93;

// The value has been changed for the copy
for (int tmpValue : valueCopyArray) {
    System.out.println(tmpValue);
}

// The original array retains the original value
System.out.println(explicitLengthArray[1]);
```

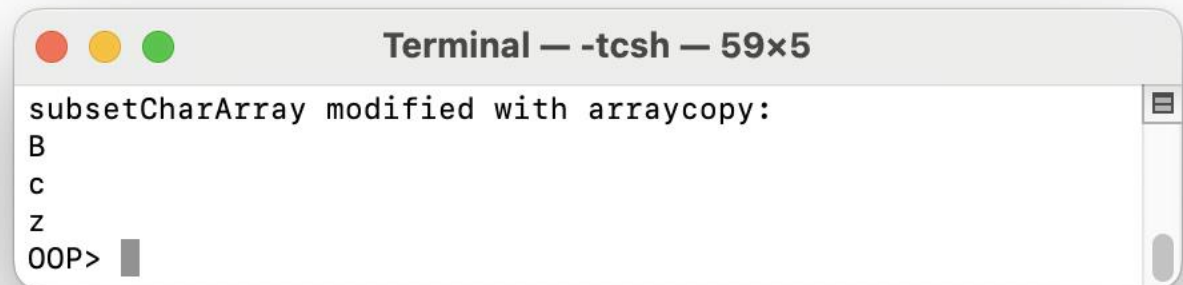
Array Copies

```
import java.util.Arrays;  
  
char[] originalCharArray = {'a', 'B', 'c', 'D', '5'};  
  
char[] subsetCharArray = Arrays.copyOfRange(originalCharArray, 1, 4);
```



Array Copies

```
... Arrays.fill(subsetCharArray, 'z');  
  
// From originalCharArray, starting at index 1, copy to subsetCharArray  
// starting at index 0. Copy 2 elements.  
System.arraycopy(originalCharArray, 1, subsetCharArray, 0, 2);  
  
for (char tmpValue : subsetCharArray) {  
    System.out.println(tmpValue);  
}  
...
```



A terminal window titled "Terminal — -tcsh — 59x5" showing the output of the Java code. The output is "subsetCharArray modified with arraycopy:" followed by the characters "B", "c", and "Z" on separate lines. The prompt "OOP>" is visible at the bottom.

```
subsetCharArray modified with arraycopy:  
B  
c  
Z  
OOP>
```

Exercise 8.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Multidimensional Arrays

Think of a two-dimensional array as a table, with rows and columns

Each element in the first array contains a reference to another array

- Each row (array) can have a different number of columns (elements)
- The leftmost index indicates the outermost array

It is possible to have N dimensions of arrays

Multidimensional Arrays

```
... // Declaring a two-dimensional array
String nested[][] = { {"Apple", "Bread", "Corn"}, {"Aardvark", "Buffalo"} };

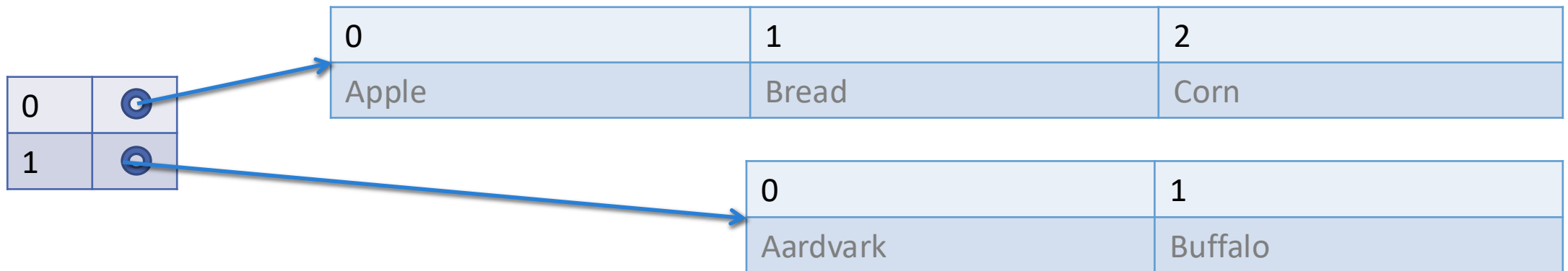
// Iterating over the arrays
for(int i=0; i < nested.length; i++) {
    for(int j=0; j < nested[i].length; j++) {
        System.out.println("nested[" + i + "][" + j + "] " + nested[i][j]);
    }
}
...
```

```
nested[0][0] Apple
nested[0][1] Bread
nested[0][2] Corn
nested[1][0] Aardvark
nested[1][1] Buffalo
```

Multidimensional Arrays

```
// Declaring a two-dimensional array
String nested[][] = { {"Apple", "Bread", "Corn"}, {"Aardvark", "Buffalo"} };

// Iterating over the arrays
for(int i=0; i < nested.length; i++) {
    for(int j=0; j < nested[i].length; j++) {
        System.out.println("nested[" + i + "][" + j + "] " + nested[i][j]);
    }
}
```



Multidimensional Arrays

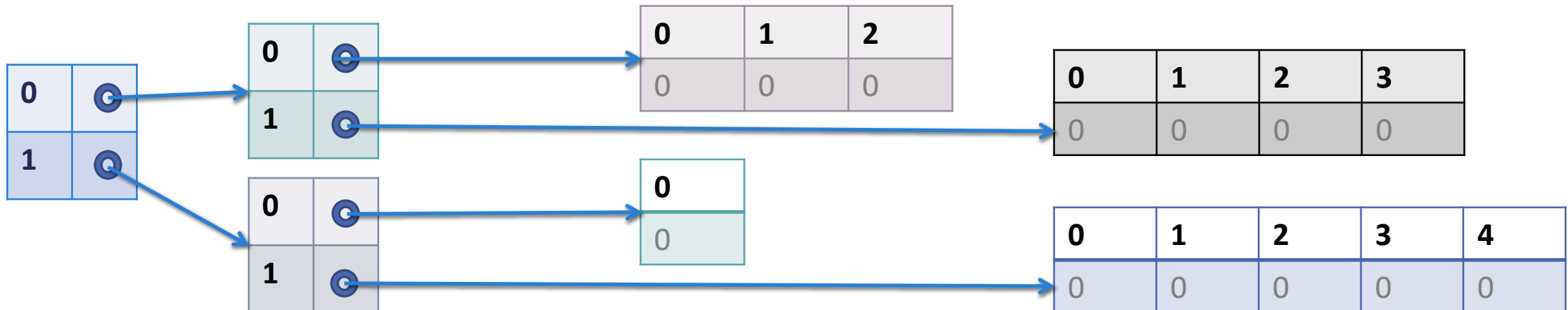
02_MultiDimensional

```
// Demonstrate changing the second value of the first "column"
    nested[0][1] = "Banana";
    for(int i=0; i < nested.length; i++) {
        for(int j=0; j < nested[i].length; j++) {
            System.out.println("nested[" + i + "][" + j + "] " +
nested[i][j]);
        }
    }
```

```
nested[0][0] Apple
nested[0][1] Banana
nested[0][2] Corn
nested[1][0] Aardvark
nested[1][1] Buffalo
```

Multidimensional Arrays

```
... int intNested[][][];  
// Outermost array length is 2; each element references an array of length 2  
intNested = new int[2][2][];  
  
// Set the length of each of the arrays which contains ints  
intNested[0][0] = new int[3];  
intNested[0][1] = new int[4];  
intNested[1][0] = new int[1];  
intNested[1][1] = new int[5];  
...
```



Multidimensional Arrays

```
intNested[0][0] = new int[3];  
intNested[0][1] = new int[4];  
intNested[1][0] = new String[1];  
intNested[1][1] = new String[5];
```

This code will not work!

Exercise 8.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Properties of arrays

Default values in declared arrays

Summary: Skills

Declare and initialize an array for any primitive data type, or objects

Use a foreach style loop to iterate over an array

Use a for loop to iterate over an array

Copy an array, by reference or value

Use common array manipulation methods

Creating and accessing multidimensional arrays

Additional Information

Related reading

- Volume 1, Chapter 3.10: Arrays